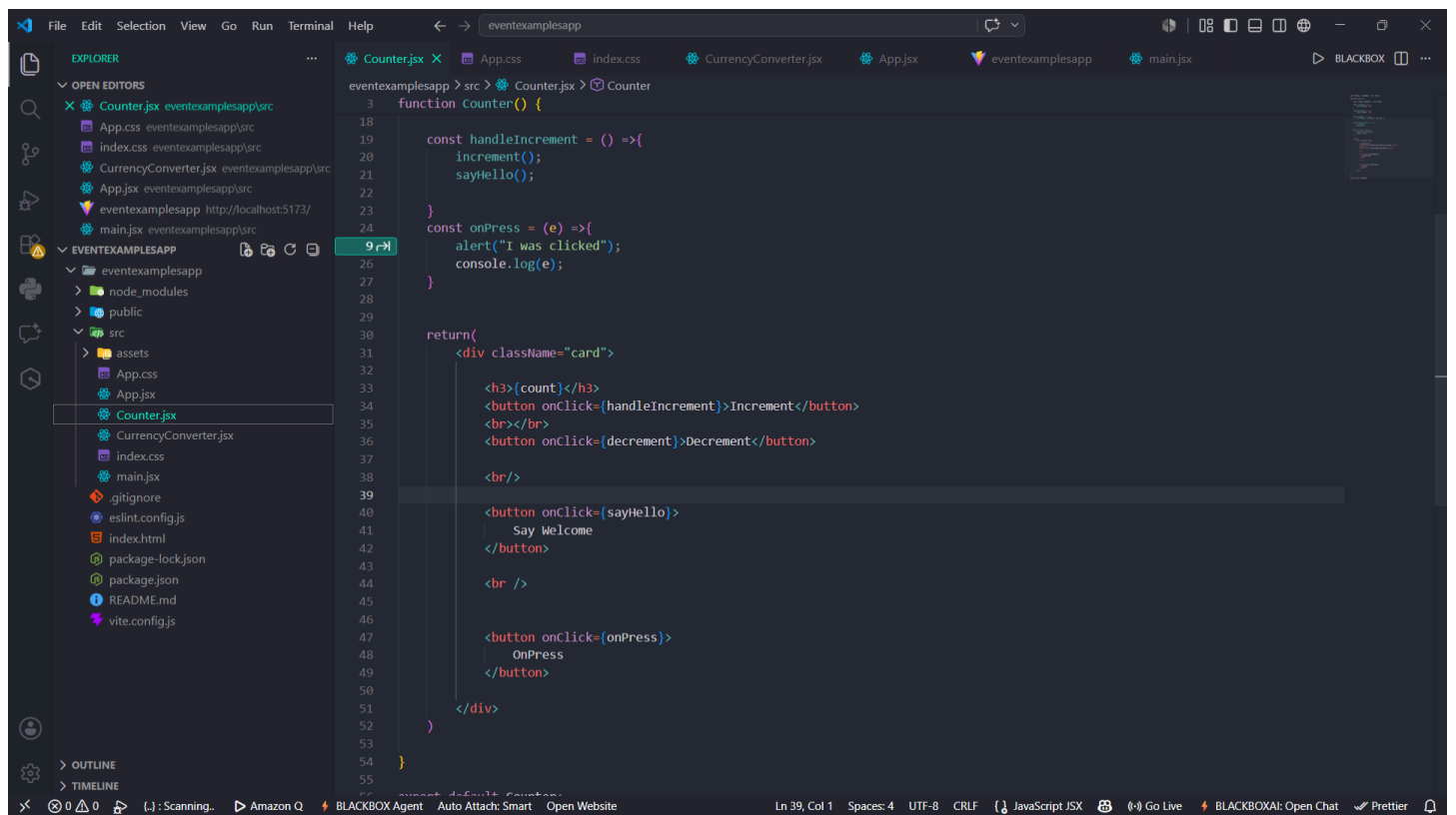


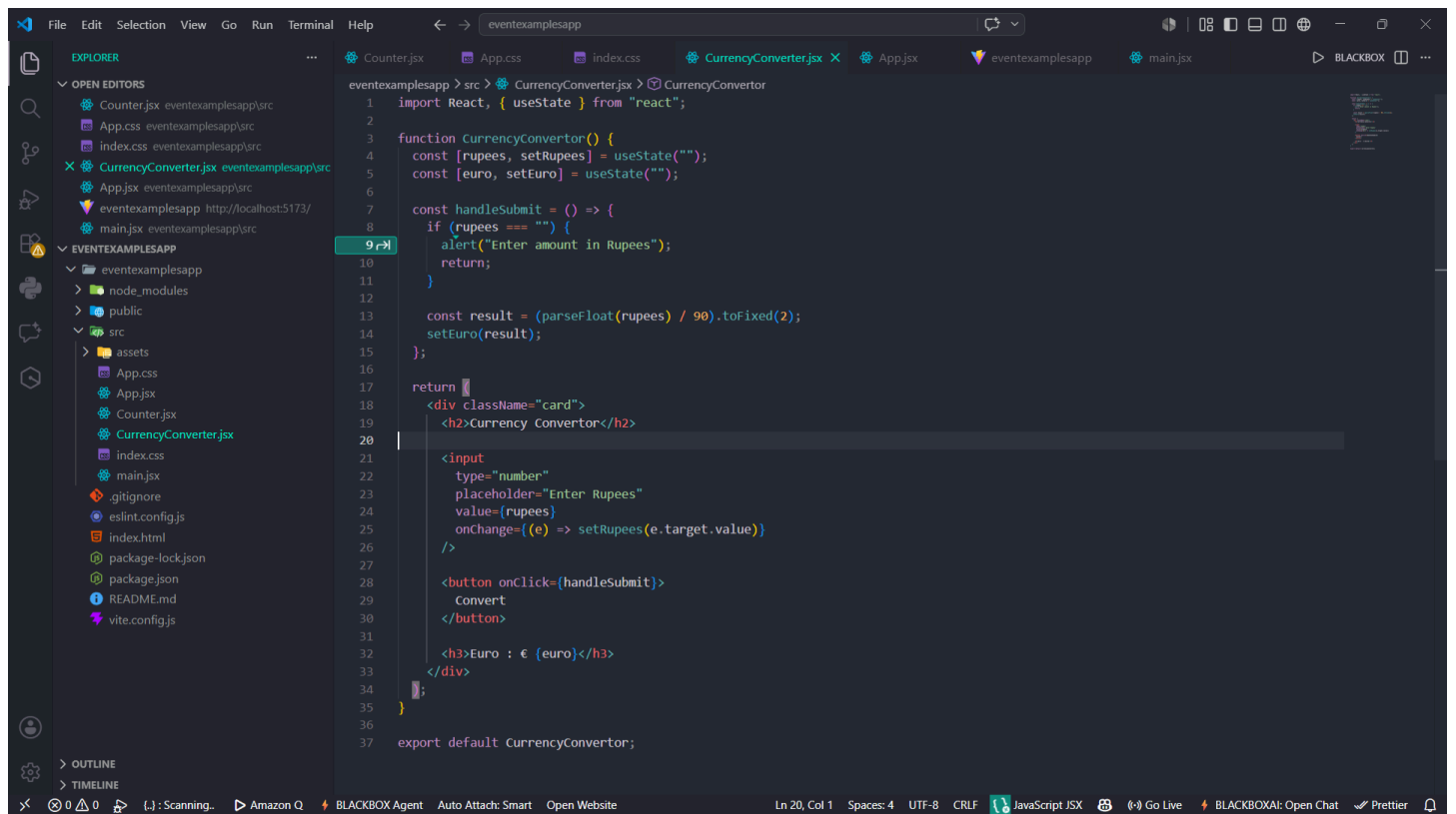
Code for Counter.jsx



The screenshot shows the VS Code editor with the file Counter.jsx open. The Explorer sidebar on the left shows the project structure for eventexamplesapp, with Counter.jsx selected. The main editor displays the following code:

```
function Counter() {  
  18  
  19  
  20    const handleIncrement = () =>{  
  21      increment();  
  22      sayHello();  
  23    }  
  24  
  25    const onPress = (e) =>{  
  26      alert("I was clicked");  
  27      console.log(e);  
  28    }  
  29  
  30    return(  
  31      <div className="card">  
  32  
  33        <h3>{count}</h3>  
  34        <button onClick={handleIncrement}>Increment</button>  
  35        <br></br>  
  36        <button onClick={decrement}>Decrement</button>  
  37  
  38        <br/>  
  39  
  40        <button onClick={sayHello}>  
  41          Say Welcome  
  42        </button>  
  43  
  44        <br />  
  45  
  46        <button onClick={onPress}>  
  47          OnPress  
  48        </button>  
  49  
  50      </div>  
  51    )  
  52  }  
  53  
  54  
  55  
  56  export default Counter;  
}
```

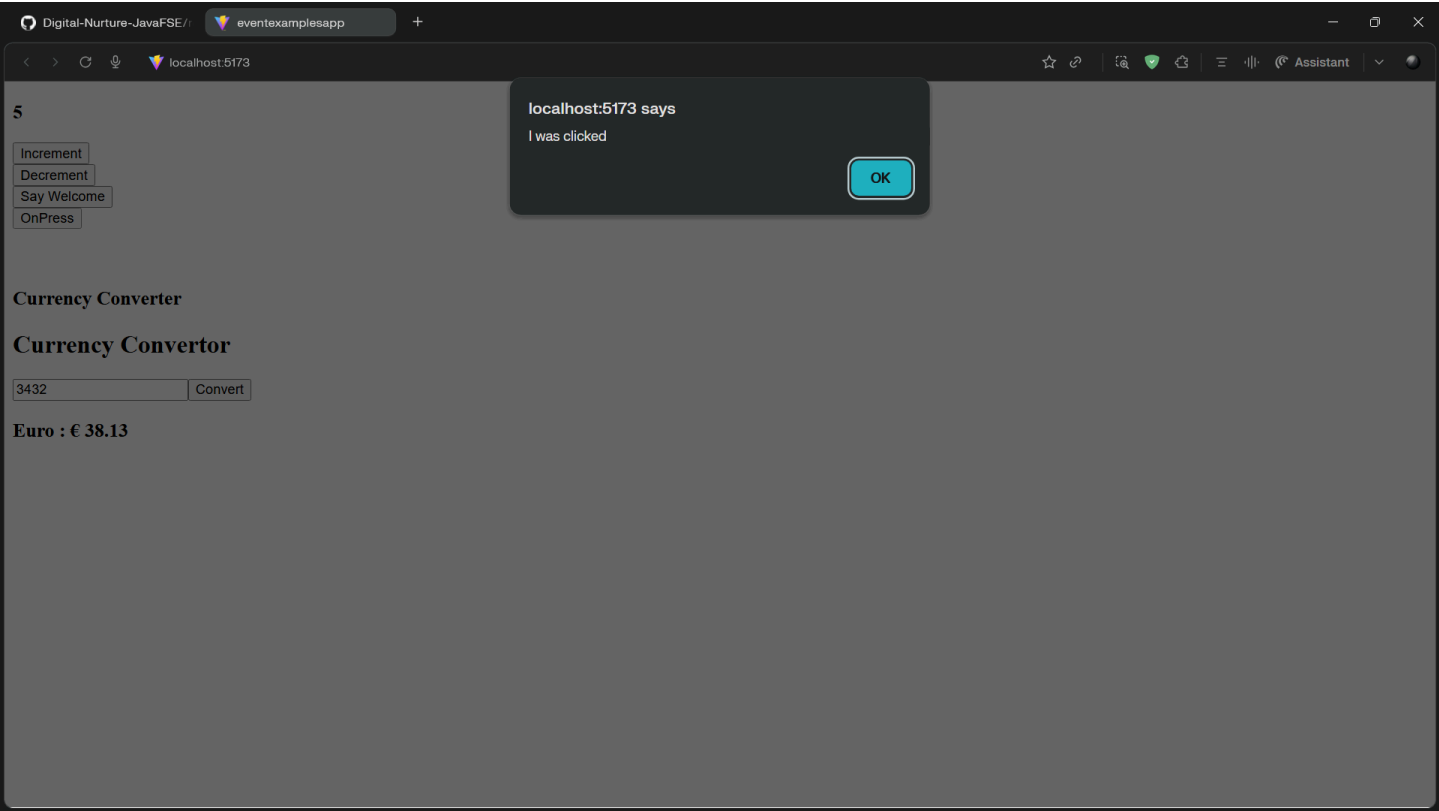
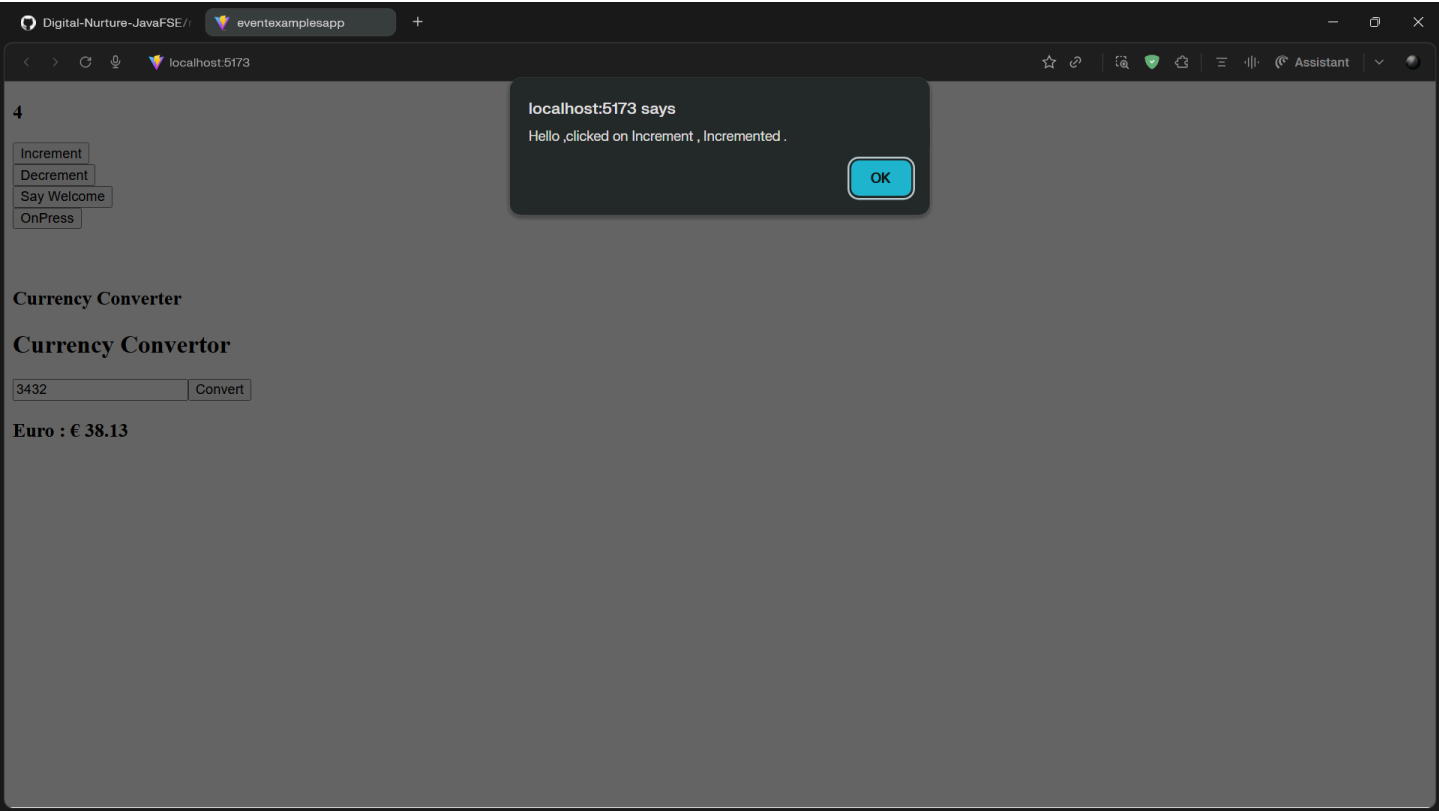
CurrencyConverter.jsx



The screenshot shows the VS Code editor with the file CurrencyConverter.jsx open. The Explorer sidebar on the left shows the project structure for eventexamplesapp, with CurrencyConverter.jsx selected. The main editor displays the following code:

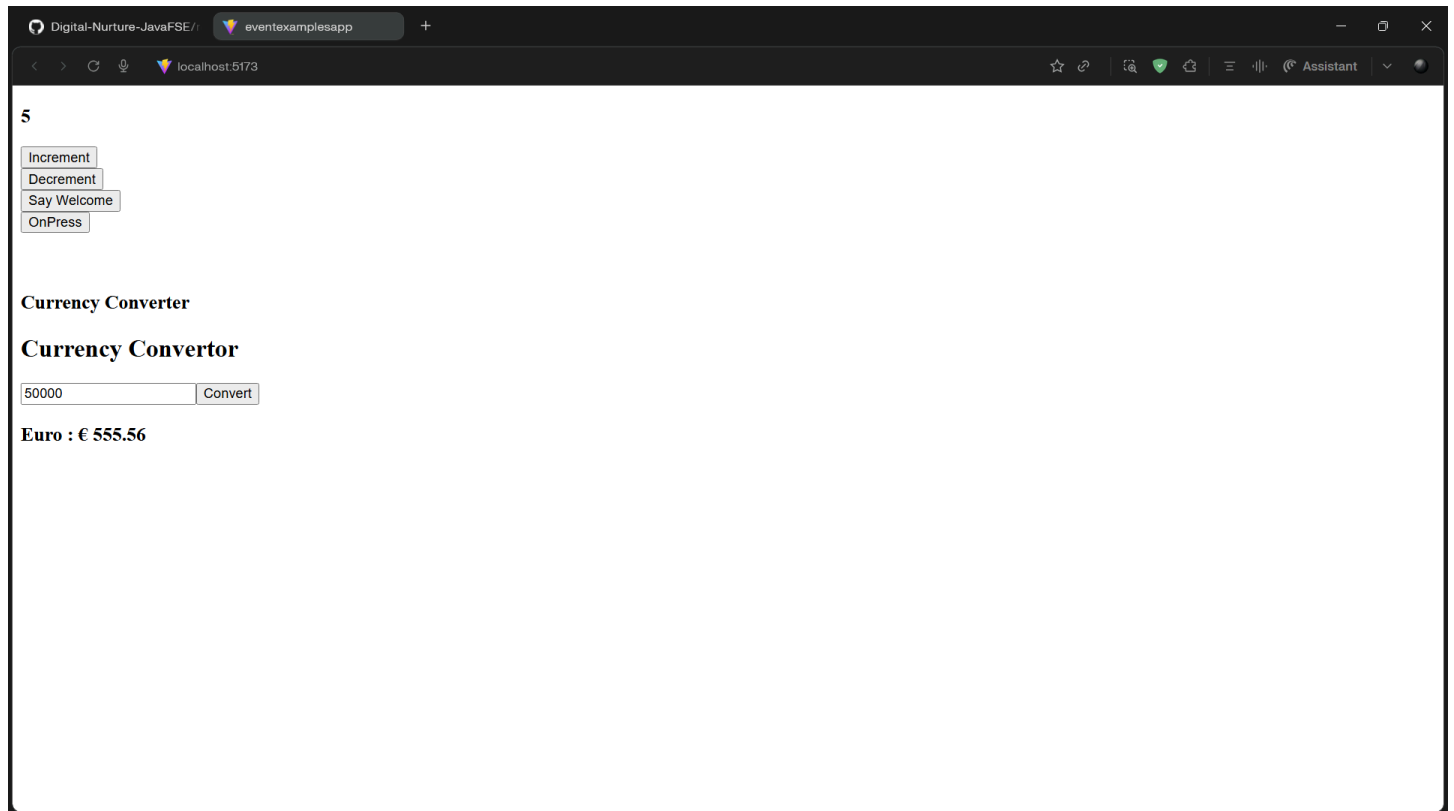
```
1  import React, { useState } from "react";  
2  
3  function CurrencyConvortor() {  
4    const [rupees, setRupees] = useState("");  
5    const [euro, setEuro] = useState("");  
6  
7    const handleSubmit = () => {  
8      if (rupees === "") {  
9        alert("Enter amount in Rupees");  
10       return;  
11     }  
12  
13     const result = (parseFloat(rupees) / 90).toFixed(2);  
14     setEuro(result);  
15   };  
16  
17   return (  
18     <div className="card">  
19       <h2>Currency Convortor</h2>  
20  
21       <input  
22         type="number"  
23         placeholder="Enter Rupees"  
24         value={rupees}  
25         onChange={(e) => setRupees(e.target.value)}  
26       />  
27  
28       <button onClick={handleSubmit}>  
29         Convert  
30       </button>  
31  
32       <h3>Euro : € {euro}</h3>  
33     </div>  
34   );  
35 }  
36  
37 export default CurrencyConvortor;
```

Output for Increment , Conversion



Output

Currency convert



Objectives --->

1. Explain React Events

React events are actions or occurrences triggered by user interactions, such as clicking a button, typing in a text field, submitting a form, or moving the mouse. React uses an event system to handle these interactions and execute the appropriate functions. React events are similar to JavaScript DOM events but are implemented using React's Synthetic Event system for consistent behavior across different browsers.

2. Explain Event Handlers

Event handlers are functions that are executed when a specific event occurs in a React application. They define the action to be performed in response to user interactions, such as updating the component state, displaying messages, validating forms, or performing calculations. Event handlers make React applications interactive and dynamic.

3. Define Synthetic Event

A Synthetic Event is React's cross-browser wrapper around the native browser event. It provides a consistent interface for handling events across all browsers while maintaining the same functionality as native events. Synthetic Events improve compatibility, performance, and simplify event handling in React applications.

4. Identify React Event Naming Convention

React follows a **camelCase** naming convention for event names. Event names begin with the prefix "**on**" followed by the event name with the first letter capitalized. Event handlers are passed as JavaScript functions rather than strings. This naming convention distinguishes React events from standard HTML event attributes.

Examples of React event names:

- onClick
- onChange
- onSubmit
- onMouseOver
- onKeyDown
- onDoubleClick

These conventions ensure consistency and readability in React applications.